

Roteiro Aula 01 de Python – UC3

Parte 1: Instalação e Preparação do Ambiente

Antes de digitar nossa primeira linha de código, precisamos instalar o interpretador do Python e escolher nossa ferramenta de trabalho (IDE).

1. Instalação do Python

1. Acesse o site oficial: python.org/downloads.
2. O site identificará seu sistema operacional automaticamente. Baixe a versão mais recente.
3.  **ATENÇÃO NA INSTALAÇÃO (CRÍTICO):** Ao abrir o instalador, certifique-se de marcar a caixa **"Add python.exe to PATH"** na primeira tela antes de clicar em *Install Now*. Isso garante que você consiga executar o Python de qualquer lugar do seu terminal.

2. Escolhendo sua IDE: PyCharm vs. VS Code

Como desenvolvedores, usamos Ambientes de Desenvolvimento Integrados (IDEs) para facilitar nosso trabalho. Vamos ver as duas principais opções:

Opção A: PyCharm (A mais completa para Python)

- Baixe a versão **Community** (gratuita) em: jetbrains.com/pycharm/download/.
- **Primeiros Passos na Interface:**
 - **Criar Projeto:** Abra o PyCharm, clique em *New Project*, escolha o local da pasta e clique em *Create*. Ele configurará um ambiente isolado (Virtualenv) automaticamente.
 - **Alterar o Visual e Fonte:** Vá em File  Settings (ou Ctrl + Alt + S)  Appearance & Behavior para mudar o tema (Dark/Light). Em Editor  Font, você pode aumentar o tamanho da letra para melhorar a leitura.
 - **Gerenciar Arquivos:** Na barra lateral esquerda (*Project*), clique com o botão direito na pasta do projeto  New  Python File para criar seus scripts .py.

Opção B: VS Code (Leve e Versátil)

Se preferir o Visual Studio Code, siga estes ajustes essenciais para Python:

1. Instale a extensão oficial **Python (da Microsoft)** através do menu de extensões (Ctrl + Shift + X). Ela habilita o *IntelliSense* (auto-completar inteligente).
2. **Configuração Recomendada:** No menu de configurações (Ctrl + ,), pesquise por Compact Folders e **desative** essa opção. Isso impede que o VS Code agrupe pastas vazias em uma única linha, facilitando a visualização da estrutura de pacotes do seu projeto.

Parte 2: O Laboratório Inicial (A IDLE do Python)

Antes de criar arquivos complexos, abra o terminal do seu computador ou a aplicação **IDLE** (que vem instalada junto com o Python).

A IDLE funciona como um "laboratório em tempo real" (REPL). Cada linha digitada é processada imediatamente. Use-a para testar os conceitos rápidos que veremos a seguir.

Parte 3: Fundamentos Técnicos Essenciais

Para implementar qualquer regra de negócio no futuro, você precisa dominar estes 6 pilares da linguagem:

1. Variáveis e Tipos Primitivos

Em Python, criamos variáveis associando um nome a um valor usando o operador `=`.

Python

Strings (Texto) - str

usuario_email = "instrutor@escola.com.br"

Inteiros (Números inteiros) - int

tentativas_login = 3

Decimais (Ponto Flutuante) - float

tempo_sessao_minutos = 15.5

Booleanos (Verdadeiro ou Falso) - bool

m2f_habilitado = True

2. Coerção, Concatenação e Interpolação

- **Coerção (Conversão de tipos):** `str()`, `int()`, `float()`, `bool()`.
- **Interpolação moderna (f-strings):** A forma mais elegante de misturar texto e variáveis.

Python

Coerção: Convertendo a entrada de texto para booleano ou inteiro

```
input_seguranca = "1"
```

```
status_m2f = bool(int(input_seguranca)) # Transforma "1" em 1, e depois em True
```

f-strings (Uso recomendado)

```
print(f"O status do M2F para o usuário é: {status_m2f}")
```

3. Operadores Aritméticos e Precedência

Python segue a regra matemática padrão de precedência (Parênteses primeiro, depois multiplicação/divisão, depois adição/subtração).

- `+` (Adição), `-` (Subtração), `*` (Multiplicação), `/` (Divisão real), `//` (Divisão inteira).

4. Captura de Dados (input)

Toda informação capturada via `input()` entra no programa estritamente como texto (`str`). Se precisar de números, deve aplicar a coerção.

Python

```
cpf_digitado = input("Digite seu CPF (apenas números): ")
```

5. Condicionais e Operadores Lógicos

Aqui mora o coração da lógica de autenticação exigida no mercado.

- **Operadores de Comparação:** `==` (Igual), `!=` (Diferente).
- **Operadores Lógicos:** `and` (Exige que ambas as condições sejam verdadeiras), `or` (Exige que pelo menos uma seja verdadeira), `not` (Inverte o valor booleano).
- **Operadores de Associação:** `in` (Verifica se um elemento está contido em uma coleção/lista).



Parte 4: Caderno de Exercícios (UC1 - Consolidação)

Para fixar este conteúdo, desenvolva os scripts abaixo utilizando o PyCharm ou VS Code. Eles simulam a base estrutural de um sistema de login com regras estritas.

Exercício 1: O Verificador de Login (CPF + E-mail)

Objetivo: Solicitar os dados de cadastro de um usuário fictício e, em seguida, simular uma tentativa de login. O login só será bem-sucedido se o CPF estiver correto **E** o e-mail digitado for igual ao e-mail principal **OU** ao e-mail secundário.

Exercício 2: O Direcionador de Fluxo (M2F Opcional)

Objetivo: Com o usuário já autenticado no passo anterior, verifique se ele possui a autenticação de dois fatores (M2F) habilitada. Exiba mensagens customizadas baseadas no estado da configuração.

Soluções Comentadas das Atividades

Aqui está o código padrão que resolve ambos os cenários de forma limpa, aplicando estritamente os conceitos de condicionais (if/elif/else), operadores lógicos (and, or), operadores de associação (in) e f-strings.

Python

```
#
=====

# RESOLUÇÃO DO EXERCÍCIO 1 & 2: SISTEMA DE LOGIN E DIRECIONAMENTO
M2F

#
=====

# 1. Base de dados simulada (Dados cadastrados no sistema)

banco_cpf = "12345678900"

banco_email_principal = "diretoria@escola.com"

banco_email_secundario = "suporte@escola.com" # Segundo e-mail opcional

banco_m2f_enabled = True # Define se o usuário escolheu usar M2F (True/False)

print("--- TELA DE AUTENTICAÇÃO ---")
```

2. Entrada de dados do usuário remoto

```
input_cpf = input("Informe o CPF cadastrado: ")
```

```
input_email = input("Informe o e-mail de acesso: ")
```

3. Processamento das Regras de Validação (Lógica solicitada na especificação)

```
cpf_valido = input_cpf == banco_cpf
```

O e-mail informado deve corresponder ao principal OU ao secundário

```
email_valido = (input_email == banco_email_principal) or (input_email ==  
banco_email_secundario)
```

O login exige obrigatoriamente CPF E E-mail corretos simultaneamente

```
pode_logar = cpf_valido and email_valido
```

4. Saída de Dados e Validação de Fluxo de Segurança

if pode_logar:

```
print("\n✅ Credenciais básicas validadas com sucesso.")
```

Verificação do status de M2F (Caso de Uso de Fluxo Opcional)

if banco_m2f_enabled:

Saída esperada se o M2F estiver ativo para este usuário específico

```
print("🔒 [M2F ATIVO] - Segunda etapa de autenticação é obrigatória.")
```

```
print("Resultado retornado: {'authenticated': False, 'm2fRequired': True}")
```

else:

Saída esperada se o M2F estiver desativado

```
print("🔓 [M2F INATIVO] - Acesso direto liberado para o painel.")
```

```
print("Resultado retornado: {'authenticated': True, 'm2fRequired': False}")
```

else:

```
print("\n ❌ Falha na autenticação: CPF ou E-mail incorretos.")
```

Desafio Final: Subindo para o GitHub

Para fechar nossa primeira aula com padrões reais do mercado de trabalho, salve o arquivo que você criou e envie-o para o seu portfólio.

1. Crie uma conta em github.com se ainda não tiver uma.
2. Crie um repositório público chamado curso-python-fundamentos.
3. Abra o terminal na pasta do seu projeto e execute os comandos:

Bash

```
git init
```

```
git add .
```

```
git commit -m "Feat: Implementação da lógica inicial de login e validação M2F"
```

```
git branch -M main
```

```
git remote add origin URL_DO_SEU_REPOSITORIO
```

```
git push -u origin main
```